

# LAB 4

## LLM OBSERVABILITY, MONITORING, AND EVALUATION ON PRODUCTION

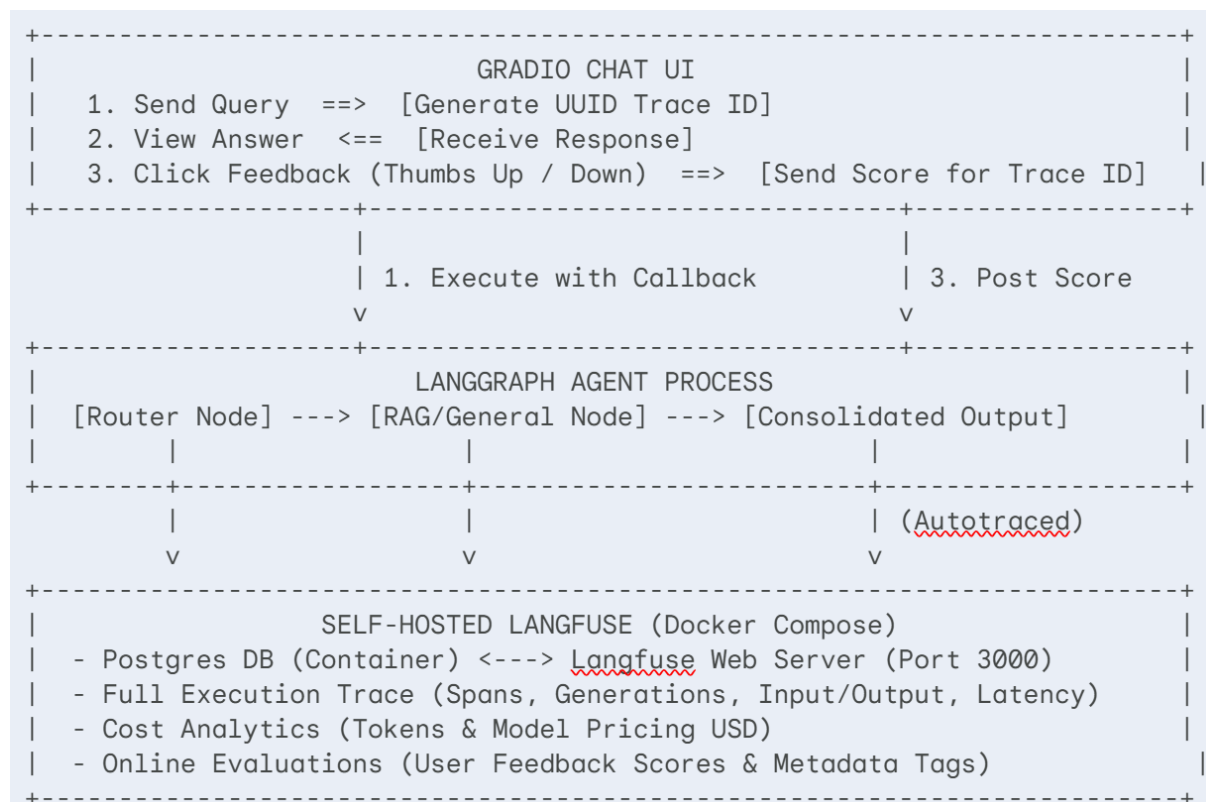
|               |                                 |
|---------------|---------------------------------|
| Course        | DDM501                          |
| Weight        | 15%                             |
| Format        | Team Lab (3-4 members per team) |
| Prerequisites | Labs 1, 2, and 3 completed      |

# 1. OVERVIEW

## 1.1. Introduction

Building an AI agent is only the first step. Bringing an agentic workflow to production requires continuous monitoring, debugging, and quality evaluation.

In this lab, you will extend the **Multi-Agent Chatbot** created in **Lab 3** by integrating and self-hosting **Langfuse**—an open-source LLM engineering platform. This lab simulates the role of an AI Platform / MLOps Engineer who must establish private, self-hosted observability pipelines. You will learn to spin up Langfuse services locally using Docker Compose, trace multi-step agent decisions, track detailed pricing and token metrics, and establish online evaluation loops using human-in-the-loop feedback mechanisms.



## 1.2. Scenario: Operationalizing the Enterprise Support Chatbot

The Multi-Agent Chatbot you developed in Lab 3 has been released to beta testers. However, the product team is struggling with several operational challenges:

- Debugging Failures:** Some user queries about policy specifications are being routed to the General Agent instead of the RAG Agent, resulting in incomplete responses, but engineers can't see why the router failed.
- Cost Management & Data Sovereignty:** The CFO wants a detailed breakdown of LLM expenses. Additionally, due to strict corporate privacy compliance, no telemetry or tracing data is allowed to leave internal networks. You cannot use a public SaaS cloud.
- Quality Assurance:** There is no mechanism to collect feedback or run evaluation metrics on live outputs to see if the RAG agent is hallucinating.

Your team's task is to containerize and spin up a local, private instance of Langfuse using Docker Compose, instrument your LangGraph application to stream traces to it, and build a UI mechanism to capture user feedback scores.

## 2. BACKGROUND

### 2.1. Self-Hosted LLM Observability Architecture

While SaaS tools are convenient, enterprise settings often require hosting monitoring stacks internally for compliance, security, and data sovereignty. A robust self-hosted Langfuse stack consists of:

- **Application UI & Web server (Langfuse Container):** Handles dashboard rendering, API endpoints for ingestion, and authentication.
- **Database (PostgreSQL Container):** Stores persistent telemetry, traces, user sessions, models metadata, and dynamic evaluation scores.
- **Ingestion API (Port 3000):** Consumes structured JSON SDK payloads asynchronously from your application runtime.

### 2.2. LLM Observability Tracing

Standard application logging fails in agentic systems because LLM calls are non-deterministic, nested, and asynchronous. **Observability Tracing** models executions as nested trees:

- **Trace:** The overall journey of a single request (e.g., a single user question in Gradio).
- **Span:** A specific period of execution representing a discrete step (e.g., the execution of the LangGraph router node).
- **Generation:** A specialized span representing a single call to an LLM. It records the prompt, generation parameters, output completions, model versions, and latency.

### 2.3. Cost Tracking & Analytical Tagging

LLM APIs charge based on input and output token counts. Langfuse monitors these metrics automatically by reading token usage from response metadata and matching model configurations to a local pricing table to calculate the exact cost of each trace in USD. Custom variables like `user_id`, `session_id`, and `routing_category` can be bound to target cohorts.

## 3. HANDS-ON GUIDES

### Task 1: Deploy Self-Hosted Langfuse via Docker Compose

To ensure data privacy, you will host Langfuse locally inside isolated docker containers.

#### 1.1. Create Docker Compose Configuration

In your project root, create a file named `docker-compose.yml`:

```
version: '3.8'
```

```
services:
```

```
  db:
```

```
    image: postgres:16-alpine
```

```
    container_name: langfuse-postgres
```

```
    restart: always
```

```
    environment:
```

POSTGRES\_USER: postgres  
POSTGRES\_PASSWORD: mysecurepassword  
POSTGRES\_DB: langfuse

ports:

- "5432:5432"

volumes:

- pgdata:/var/lib/postgresql/data

langfuse:

image: langfuse/langfuse:2

container\_name: langfuse-web

restart: always

depends\_on:

- db

ports:

- "3000:3000"

environment:

- DATABASE\_URL=postgresql://postgres:mysecurepassword@db:5432/langfuse

- NEXTAUTH\_SECRET=my\_super\_secret\_nextauth\_key\_at\_least\_32\_characters\_long

- NEXTAUTH\_URL=http://localhost:3000

- TELEMETRY\_ENABLED=false

- LANGFUSE\_INIT\_PROJECT\_NAME=DDM501-Enterprise-Support

# Enables standard email/password login configuration

- LANGFUSE\_ENABLE\_SIGN\_UP=true

volumes:

pgdata:

### *1.2. Run the Stack*

Start the containers using Docker:

`docker-compose up -d`

Verify the services are running by accessing the UI in your browser at <http://localhost:3000>.

### *1.3. Generate Local Credentials*

- Open <http://localhost:3000> and click **Sign Up** to create an admin account.
- Navigate to your project settings and click **Create API Credentials**.
- Copy your local credentials. Note that your LANGFUSE\_HOST must point to your local port 3000.

### *1.4. Update Environment Configuration*

Configure your .env file in the project root:

OPENAI\_API\_KEY=your-openai-key

LANGFUSE\_PUBLIC\_KEY=pk-lf-your-local-public-key

```
LANGFUSE_SECRET_KEY=sk-lf-your-local-secret-key
LANGFUSE_HOST=http://localhost:3000
```

Update your app/config.py to parse these variables using python-dotenv.

## Task 2: Instrument LangGraph with Langfuse Tracing

### 2.1. Add the Langfuse Callback Handler

Langfuse integrates natively with LangChain and LangGraph via the CallbackHandler. You will:

- Import CallbackHandler from the langfuse.callback library.
- Initialize the callback handler pointing to your local self-hosted endpoint.
- Pass this handler to the execution context when invoking your compiled LangGraph app.

### 2.2. Map Node Executions to Langfuse Spans

By passing the callback to the graph execution config, LangGraph will automatically register each transition as a step. Ensure that your LangChain-based models (e.g., ChatOpenAI) inside app/graph.py are properly utilizing the context so that nested API calls are recorded as **Generations**.

## Task 3: Implement Metadata Logging and Cost Monitoring

To make your dashboard actionable, you must attach metadata to the trace on every interaction. You will update the invocation script to append:

- user\_id: A static string or hashed identifier indicating the active user.
- session\_id: A UUID persistent across a single Gradio tab session.
- tags: A list indicating which version of the routing model is active (e.g., ["v1-router-gpt-4o-mini"]).

## Task 4: Implement Online Feedback inside Gradio

### 4.1. Capture and Track Trace IDs

To submit feedback on a specific response, the web interface must know the unique **Trace ID** of that interaction.

- a) When generating a response, instantiate the Langfuse CallbackHandler with a custom pre-generated trace\_id (a UUID).
- b) Store this trace\_id in a Gradio gr.State variable.

### 4.2. Create Feedback UI Components

Add interactive feedback buttons (e.g., Thumbs Up 👍 / Thumbs Down 👎) to your Gradio interface:

- a) When the user clicks 👍, call a backend Python function that logs a score of 1.0 (helpfulness) to Langfuse using the saved trace\_id.
- b) When the user clicks 👎, log a score of 0.0.

## 4. DELIVERABLES & GRADING

### 4.1. Deliverables

Your team must submit a GitHub repository containing:

- a) **Docker Compose Stack:** A verified docker-compose.yml that boots up the Langfuse web

server and PostgreSQL.

- b) **Instrumented Source Code:** Your completed LangGraph and Gradio implementation incorporating the Langfuse Callback Handler and Score feedback calls pointed to the self-hosted instance.
- c) **Comprehensive Test Suite (tests/test\_observability.py):** At least 4 automated tests verifying:
  - a. Traces are correctly generated with valid UUID formats.
  - b. Feedback operations handle empty or missing trace IDs gracefully.
  - c. Connection limits and host validation checks are configured.
- d) **Observability Project Export:** A generated PDF containing:
  - a. A screenshot of your self-hosted Langfuse workspace dashboard running on localhost:3000.
  - b. A screenshot of a single complete LangGraph trace, illustrating the execution path (Router  $\rightarrow$  Specialist Node).
  - c. A screenshot of the **Scores Table** displaying recorded user feedback values (1.0 and 0.0).
- e) **README.md Documentation:** Updated system execution procedures, local credential configurations, and Docker startup commands.

## 4.2. Grading Rubric

| Criteria                             | Weight | Detailed Description                                                                                                                                                                        |
|--------------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Self-Hosted Infrastructure           | 25%    | Correct docker-compose.yml structure with persistent PG volume mapping (10%). Successful local workspace registration and API keys generation on local Port 3000 (15%).                     |
| Langfuse Integration & Graph Tracing | 20%    | Correct initialization of local CallbackHandler pointing to custom LANGFUSE_HOST (5%). Traces accurately streamed on query (5%). Graph transitions rendered as separate nested spans (10%). |
| Cost Tracking & Analytical Metadata  | 20%    | Local pricing tables metadata working, computing token usage in USD (5%). Custom analytics tags parsed: session_id, user_id, and route_taken (15%).                                         |

|                                                 |            |                                                                                                                                                                                                                  |
|-------------------------------------------------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Online Feedback Integration (Evaluation)</b> | <b>25%</b> | Correct capture of UUID Trace IDs in the frontend interface (10%). Active UI interaction (👍 / 🗑️ buttons) successfully logs evaluation scores directly to self-hosted Langfuse on the corresponding trace (15%). |
| <b>Testing &amp; Documentation</b>              | <b>10%</b> | Docker boot instructions in README.md (5%). Unit tests covering route classification or connection boundaries written and passing (5%).                                                                          |

### 4.3. Submission Rules

- **Deadline:** 1 week after the scheduled lab session.
- **Format:** Submissions must be delivered as a single packaged .zip file containing clean source code, document data, and a valid GitHub repository URL. Do not include your .env or venv folders inside the submitted archive.